

# Dynamic Resource Allocation in Cloud Computing via Deep Reinforcement Learning

**Team Members:** Arnav Ahuja(aa5790), Tanish Patel (tp2899)

**1. Motivation and Problem Description** Cloud data centers consume an immense and growing share of global electricity. Efficiently mapping incoming computational workloads (jobs/tasks) to physical servers is a critical challenge, as this mapping has a direct, measurable impact on energy consumption. The objective of this project is to develop a dynamic resource allocation scheduler that decides in real-time which server handles each job. This intelligent scheduler aims to minimize overall power consumption while strictly adhering to Service Level Agreements (SLAs), ensuring jobs are processed within acceptable latency bounds.

**2. Why Reinforcement Learning?** Standard cloud schedulers rely on static heuristics such as Round Robin, Shortest Job First, or First-Fit Decreasing. These heuristics are rigid and fail to adapt to unpredictable, bursty workload patterns or the non-linear power dynamics of modern servers. Supervised learning is also ill-suited for this problem, as generating the "ground truth" optimal schedule for a large cluster over time is an NP-hard problem.

Reinforcement learning (RL) is uniquely equipped to handle this because it learns optimal policies through trial and error, optimizing for a long-term, delayed objective (like total daily energy cost). Furthermore, advanced RL techniques allow us to move beyond fragile reward shaping, such as manually balancing power vs. SLA coefficients, by treating this as a sequence modeling problem or by enforcing SLAs as hard mathematical constraints with formal guarantees.

**3. RL Problem Formulation & Simulation Setup** We will formulate the scheduling process as a discrete-time Markov Decision Process (MDP):

- **State Space  $S_t$ :** CPU and memory utilization, server power states, and job queue characteristics.
- **Action Space  $A_t$ :** Assigning the head-of-queue job to a server  $k$  in  $\{1,2,3,\dots,N\}$
- **Data & Methodology:** We will build a custom simulation environment using Gymnasium, driven by real historical job submissions from Google Cluster Data and Alibaba Cluster Traces.

**4. Proposed RL Approaches** While our primary focus is on modeling the environment accurately, we plan to implement our agents in PyTorch and explore two distinct architectures against standard heuristics:

- **Approach 1 (Agentic RL):** A framework utilizing distinct SLA Optimization and Power Assignment agents, managed by a Supervisor Module to balance competing objectives. We will also explore using Decision Transformers to condition actions on desired SLA returns.
- **Approach 2 (Offline RL):** Training an offline policy entirely on historical data batches to avoid the risks of live exploration. This formulates the problem as a Constrained MDP (CMDP).

## 5. General Plan Going Forward

- **Weeks 1-2:** Build custom Gymnasium environment with realistic power curves and parse Google/Alibaba cluster traces into a standardized format.
- **Weeks 3-4:** Implement baseline heuristic schedulers and build the PyTorch multi-agent framework (Approach 1) and offline RL/CMDP pipeline (Approach 2).
- **Weeks 5-6:** Train models, tune hyperparameters, and run comparative experiments tracking power consumption, SLA violation rates, and latency.
- **Week 7:** Finalize statistical analysis and visualizations (training curves, utilization heatmaps) and write the final paper.